SdkApi 指导文档

当前版本	V1.20251030
更新日期	2025-10-30
文件密级	[]内部 [√]公开
文件状态	[]草稿 [√]正式发布 []正在修改

概述

Android10 以上主板支持的 API 及其说明。

读者对象

本文档主要适用于以下工程师

- 软件开发工程师
- 技术支持工程师
- 销售人员

目录

概述 1

读者对象 1

接口详细描述 8

API 导入方法 8

API 注册流程图 9

API 模块分类 9
1、初始化 API 10
1.1 绑定 SdkApi 10
1.2 获取 SdkApi 实例 11
1.3 获取 SDK 服务连接状态 12
1.4 解绑 SdkApi 13
2、设备信息 API 14
2.1 获取 API aar 库版本 14
2.2 获取 API Sdk 服务版本 15
2.3 获取主板系统版本 16
2.4 获取主板主芯片名称 17
2.5 获取主板序列号 18
2.6 获取主板内存大小 19
2.7 获取主板存储大小 20
2.8 获取 MCU 版本 21
2.9 设置系统屏幕密度 22
2.10 获取系统屏幕密度 23
2.11 设置系统 prop 属性 24
2.12 获取系统 prop 属性 25
2.13 获取主板以太网 MAC 地址 26
2.14 获取主板可用内存大小 27
2.15 获取主板可用存储大小 28
3、显示相关 API 29
3.1 设置屏幕方向 29
3.2 获取屏幕方向 30

3.3 设置屏幕开关 31
3.4 获取屏幕开关 32
3.5 设置屏幕亮度 33
3.6 获取屏幕亮度 34
3.7 设置背光取反 35
3.8 获取背光取反 36
3.9 设置背光 PWM 最大值和最小值 37
3.10 获取背光 PWM 最大值和最小值 38
3.11 设置屏幕分辨率 39
3.12 获取屏幕分辨率 40
3.13 设置 lvds 模式 41
3.14 获取 lvds 模式 42
3.15 设置 lvds Even-ODD 交换 43
3.16 获取 lvds Even-ODD 交换 44
3.17 设置 lvds 通道 45
3.18 获取 lvds 通道 46
3.19 获取支持的分辨率列表 47
3.20 设置自定义分辨率 Timing 48
3.21 设置自定义 mipi 指令 50
3.22 设置背光参数 51
3.23 获取背光参数 52
3.24 设置画面显示大小 53
3.25 获取画面显示位置 54
4、系统控制 API 55
4.1 设备重启 55

4.2 截屏 56
4.3 隐藏或者显示导航栏和状态栏 57
4.4 获取导航栏和状态栏状态 58
4.5 隐藏或者显示状态栏 59
4.6 获取状态栏状态 60
4.7 隐藏或者显示导航栏 61
4.8 获取导航栏状态 62
4.9 设备普通待机 63
4.10 获取设备待机状态 64
4.11 设备低功耗待机 65
4.12 设备低功耗定时待机 66
4.13 设置风扇开关 67
4.14 获取风扇开关状态 68
4.15 静默安装 App 69
4.16 静默安装 App 70
4.17 静默卸载 App 71
4.18 获取触摸最后一次触摸时间 72
4.19 格式化设备 73
4.20 修改开机动画音量大小 74
4.21 恢复出厂设置 75
4.22 获取开机累计时长 76
4.23 重置开机累计时长 77
4.24 获取 NTP 地址 78
4.25 重置 NTP 地址 79
5、定时开关机 API 80

5.1 设置定时开关 80
5.2 获取定时开关状态 81
5.3 设置定时开机时间 82
5.4 清除定时开机时间 83
5.5 获取定时开机时间 84
5.6 设置定时关机时间 85
5.7 清除定时关机时间 86
5.8 获取定时关机时间 87
5.9 设置定时关机模式 88
5.10 获取定时关机模式 89
6、喂狗相关 API 90
6.1 设置喂狗主体 90
6.2 获取喂狗主体方 91
6.3 设置看门狗超时时间 92
6.4 喂狗 93
7、系统时间相关 API 94
7.1 设置系统时间 94
7.2 获取 RTC 时间 95
7.3 设置自动校准时间 96
7.4 获取自动校准时间开关状态 97
8、硬件相关 API 98
8.1 获取 GPIO 输入电平 98
8.2 设置 GPIO 输出状态 99
8.3 读取 flash 数据 100
8.4 写 flash 数据 101

8.5 主板运行指示灯 102
8.6 获取主板运行指示灯 103
8.7 设置按键板红色指示灯开关 104
8.8 获取按键板红色指示灯开关 105
8.9 设置按键板绿色指示灯开关 106
8.10 获取按键板绿色指示灯开关 107
~~8.11 设置 USB 端口电源开关~~ 108
~~8.12 获取 USB 端口电源开关~~ 109
8.13 设置喇叭功率参数 110
8.14 获取喇叭功率参数 111
8.15 打开串口设备 1 112
8.16 打开串口设备 2 113
8.17 关闭串口设备 114
8.18 读串口数据 115
8.19 写串口数据 116
8.20 配置并打开 can 总线 117
8.21 打开 can 设备 118
8.22 发送 can 帧数据 119
8.23 发送 can 帧数据 120
8.24 关闭 can 设备 121
8.25 获取遥控器状态 122
8.26 设置遥控器状态 123
8.27 获取串口电源开关 124
8.28 设置串口电源开关 125
9、 其他相关 API 126

9.1 获取 Sim 卡 ICCID 号 126
9.2 获取 IMEI 号 127
9.3 执行 shell 命令 128
9.4 读取 emmc 寿命 129
10、SCALE 相关 API 130
10.1 注册 scale 监听 listener 130
10.2 注销 scale 监听 listener 131
10.3 设置 scale 执行事件 132
10.4 actionId 命令集 133
11、COMMONEVENT 相关 API 141
11.1 注册 Common 监听 listener 141
11.2 注销 Common 监听 listener 142
11.3 设置 Common 执行事件 143
11.4 ISdkCommonListener 接口说明 144
11.5 CommonEvent 使用流程 145
11.6 调用接口使用示例 146
11.7 actionId 命令集 146
API 常量 182
LOGO 文件导入错误码表 182
网络驱动类型参数表 183
GPIO 值定义 184

接口详细描述

API 导入方法

Android Studio 导入 aar 包

1、 将 aar 包【SdkApiJar.aar】复制到工程目录/app/libs 下



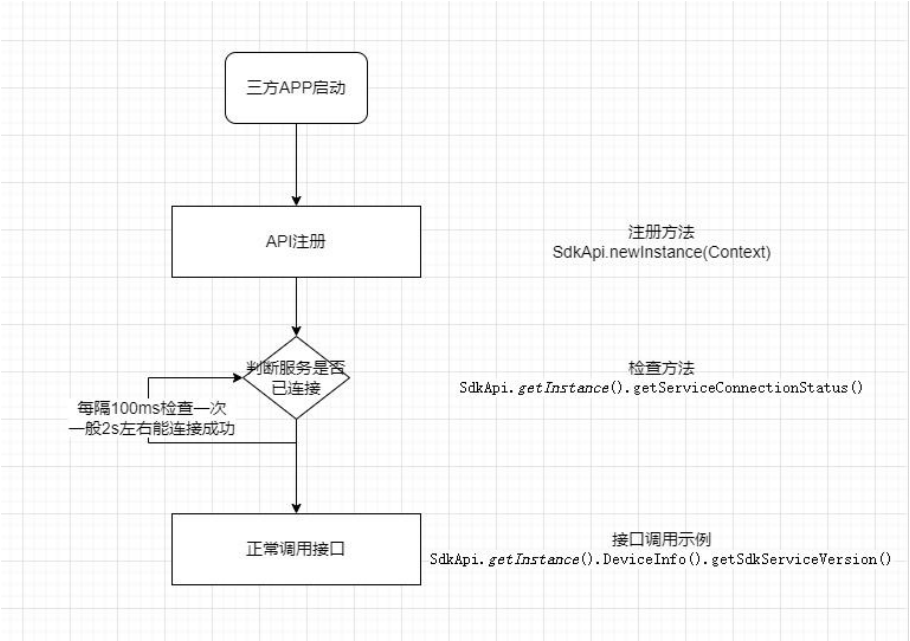
2、 在 app 下的 build.gradle 文件增加引用



3、 注册 SdkApi

4、 使用 SdkApi 相关类接口功能

API 注册流程图



API 模块分类

模块	类名称	含义
设备信息	ApiDeviceInfo	设备基本信息接口
显示控制	ApiDisplay	屏幕显示相关控制接口
系统控制	ApiSystemCtrl	系统控制相关接口
定时设置	ApiTimerOnoff	定时开关机相关接口
喂狗设置	ApiWatchdog	喂狗保活相关接口
时间设置	ApiSystemTime	系统时间相关接口
硬件控制	ApiHardware	硬件控制相关接口
其他设置	ApiOther	其他相关接口
Scale 设置	ApiScale	点屏芯片控制相关接口
通用设置	ApiCommonEvent	通用功能相关接口

1、初始化 API

1.1 绑定 SdkApi

接口名称：

void newInstance(Context context)

参数：

-- *context*：上下文

返回值：

-- 无

代码示例：

```
    SdkApi.newInstance(this);
```

备注：
-- 无

1.2 获取 SdkApi 实例

接口名称：
SdkApi getInstance()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	class	SdkApi 实例

代码示例：

```
    SdkApi api = SdkApi.getInstance();
```

备注：
-- 无

1.3 获取 SDK 服务连接状态

接口名称：
`boolean getServiceConnectionStatus()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：连接成功 false：未连接成功

代码示例：

```
boolean ret = SdkApi.getInstance().getServiceConnectionStatus();
```

备注：
-- newInstance 函数执行完毕后，需等待大概 2s 左右，服务才能正常连上

1.4 解绑 SdkApi

接口名称：
void unBindApiSdkService()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	void	

代码示例：

```
SdkApi.getInstance().unBindApiSdkService();
```

备注：
-- 解绑后，需重新绑定 SdkApi

2、 设备信息 API

-- 接口调用方式：SdkApi.getInstance().DeviceInfo().接口

2.1 获取 API aar 库版本

接口名称：

String getApiVersion()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	aar 库版本

代码示例：

```
String version =
SdkApi.getInstance().DeviceInfo().getApiVersion();

版本：YYMMdd.hhmm
示例：250813.0759
```

备注：
-- 无

2.2 获取 API Sdk 服务版本

接口名称：
`String getApiSdkServiceVersion()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	Sdk 服务版本

代码示例：

```
String version =
SdkApi.getInstance().DeviceInfo().getApiSdkServiceVersion();

版本：V2.YYMMdd.hhmmss
示例：V2.20250813.075900
```

备注：
-- 无

2.3 获取主板系统版本

接口名称：

String getFirmwareVersion()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	系统版本号

代码示例：

```
String version =
SdkApi.getInstance().DeviceInfo().getFirmwareVersion();


示例：EC3576_RK3576_Android14_A_20250813.143037
```

备注：
-- 无

2.4 获取主板主芯片名称

接口名称：

String getChipName()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	主芯片名称

代码示例：

```
String chip = SdkApi.getInstance().DeviceInfo().getChipName()
```

示例：rk3576

备注：
-- 无

2.5 获取主板序列号

接口名称：

String getSerialNumber()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	主板唯一序列号

代码示例：

```
String sn = SdkApi.getInstance().DeviceInfo().getSerialNumber()
```

示例：12345678

备注：
-- 一般是 8 位或者 21 位两种数据长度

2.6 获取主板内存大小

接口名称：
`String getDDRSize()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	内存总大小

代码示例：

```
String ddrSize = SdkApi.getInstance().DeviceInfo().getDDRSize()
```

示例：4GB

备注：
-- 无

2.7 获取主板存储大小

接口名称：

String getEMMCSize()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	总存储大小

代码示例：

```
String ddrSize = SdkApi.getInstance().DeviceInfo().getEMMCSize()
```

示例：32GB

备注：
-- 无

2.8 获取 MCU 版本

接口名称：
`void getMcuVersion(ResultCallback<ResultInfo<String>> cb)`

参数：

参数名	类型	说明
回调	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	MCU 版本号

代码示例：

```
    SdkApi.getInstance().DeviceInfo().getMcuVersion(
        new ResultCallback<ResultInfo<String>>() {
            @Override
            public void onMsg(String val) {

            }
    });
```

备注：
-- 以实际主板为准，部分主板没有 mcu，不支持此接口

2.9 设置系统屏幕密度

接口名称：
`void setLcdDensity(int density)`

参数：

参数名	类型	说明
回调	int	屏幕密度大小 值范围：100 - 480 默认：160

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().DeviceInfo().setLcdDensity(density);
```

备注：
-- 部分系统需要重启生效

2.10 获取系统屏幕密度

接口名称：
int getLcdDensity()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	int	屏幕密度大小

代码示例：

```
int density = SdkApi.getInstance().DeviceInfo().getLcdDensity()
```

备注：
-- 无

2.11 设置系统 prop 属性

接口名称：
void setProperty(String key, String value)

参数：

参数名	类型	说明
key	String	prop key 名称
value	String	prop key 对应的值

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().DeviceInfo().setProperty("persist.sys.timezone", "Asia/Shanghai")
```

备注：

-- 不支持 ro.属性的设置，android10/11 因 selinux 权限问题，建议 vendor.开头

2.12 获取系统 prop 属性

接口名称：

String getProperty(String key, String default)

参数：

参数名	类型	说明
key	String	prop key 名称
default	String	prop key 默认值

返回值：

参数名	类型	说明
参数值	String	系统 prop 属性对应的参数值

代码示例：

```
String prop = SdkApi.getInstance().DeviceInfo().getProperty(
"persist.sys.timezone", "Asia/Shanghai")
```

备注：
-- 无

2.13 获取主板以太网 MAC 地址

接口名称：
String getEthMac()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	以太网 MAC 地址，格式：”0C63FC112233”

代码示例：

```
String mac = SdkApi.getInstance().DeviceInfo().getEthMac()
```

备注：
-- 当板卡有多 mac 时，结果用@分隔，例：0C63FC112233@0C63FC445566
--> 前一个 mac：以太网 0
--> 后一个 mac：以太网 1

2.14 获取主板可用内存大小

接口名称：
`String getDDRAvailableSize()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	内存总大小

代码示例：

```
String ddrSize =
SdkApi.getInstance().DeviceInfo().getDDRAvailableSize()


示例：2.6GB
```

备注：
-- 无

2.15 获取主板可用存储大小

接口名称：

String getEMMCAvailableSize()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	总存储大小

代码示例：

```
String ddrSize =
SdkApi.getInstance().DeviceInfo().getEMMCAvailableSize()
```

示例：32.0GB

备注：
-- 无

3、 显示相关 API

-- 接口调用方式：SdkApi.getInstance().Display().接口

-- screen 显示接口 ID 包括

ID	aar 包内定义	接口值	说明
ALL	ApiTool.Screen.ALL	0	统一设置主板各个显示接口
LVDS	ApiTool.Screen.LVDS	1	主板规格书定义的 LVDS 接口
LVDS2	ApiTool.Screen.LVDS2	2	主板规格书定义的 LVDS2 接口
eDP	ApiTool.Screen.eDP	3	主板规格书定义的 eDP 接口
MIPI	ApiTool.Screen.MIPI	4	主板规格书定义的 MIPI 接口
VB1	ApiTool.Screen.VB1	5	主板规格书定义的 VB1 接口
HDMI	ApiTool.Screen.HDMI	6	主板规格书定义的 HDMI 接口
DP	ApiTool.Screen.DP	7	主板规格书定义的 DP 接口

3.1 设置屏幕方向

接口名称：
boolean setScreenRotate(int screen, int rotate)

参数：

参数名	类型	说明
screen	int	显示接口 ID
rotate	int	方向，0/90/180/270

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setScreenRotate(
ApiTool.Screen.ALL, 90)
```

备注：

-- 需重启系统生效

3.2 获取屏幕方向

接口名称：

```
void getScreenRotate(int screen, ResultCallback<ResultInfo<Integer>>
cb)
```

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	回调 val：0/90/180/270

代码示例：

```
SdkApi.getInstance().Display().getScreenRotate(ApiTool.Screen.ALL
,
    new ResultCallback<ResultInfo<Integer>>() {
        @Override
        public void onMsg(Integer val) {

        }
})
```

备注：

-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.3 设置屏幕开关

接口名称：

boolean setScreenOnoff(int screen, boolean onoff)

参数：

参数名	类型	说明
screen	int	显示接口 ID
onoff	boolean	true：打开 false：关闭

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setScreenOnoff(
ApiTool.Screen.ALL, true)
```

备注：
-- 无

3.4 获取屏幕开关

接口名称：

void getScreenOnoff(int screen, ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	true：打开 false：关闭

代码示例：

```
    SdkApi.getInstance().Display().getScreenOnoff(ApiTool.Screen.ALL,
        new ResultCallback<ResultInfo<Boolean>>() {
            @Override
            public void onMsg(Boolean val) {

            }
    })
```

备注：
-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.5 设置屏幕亮度

接口名称：
`boolean setBrightness(int screen, int brightness)`

参数：

参数名	类型	说明
screen	int	显示接口 ID
brightness	int	屏幕亮度 范围：0-100

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setBrightness(
ApiTool.Screen.ALL, 100)
```

备注：
-- 无

3.6 获取屏幕亮度

接口名称：

void getBrightness(int screen, ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	屏幕亮度值：0-100

代码示例：

```
SdkApi.getInstance().Display().getBrightness(ApiTool.Screen.ALL,
    new ResultCallback<ResultInfo<Integer>>() {
    @Override
        public void onMsg(Integer val) {

        }
})
```

备注：

-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.7 设置背光取反

接口名称：

boolean setBacklightInverse(int screen, boolean inverse)

参数：

参数名	类型	说明
screen	int	显示接口 ID
inverse	boolean	背光调整反向，默认：false false：背光电压 0-3.3/5V 或者 PWM 占空比 0-100 变化 true：背光电压 3.3/5-0V 或者 PWM 占空比 100-0 变化

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setBacklightInverse(
ApiTool.Screen.ALL, false)
```

备注：

-- 建议使用新接口 setBacklightParams

3.8 获取背光取反

接口名称：

void getBacklightInverse(int screen, ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	true：打开 false：关闭

代码示例：

```
SdkApi.getInstance().Display().getBacklightInverse(
    ApiTool.Screen.ALL,
    new ResultCallback<ResultInfo<Boolean>>() {
        @Override
        public void onMsg(Boolean val) {

        }
})
```

备注：

-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态
-- 建议使用新接口 getBacklightParams

3.9 设置背光 PWM 最大值和最小值

接口名称：

```
boolean setBacklightPWM(int screen, int min, int max)
```

参数：

参数名	类型	说明
screen	int	显示接口 ID
min	int	范围：0-100
max	int	范围：0-100

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setBacklightPWM(
ApiTool.Screen.ALL, 0, 0)
```

备注：
-- 最小值 > 最大值时，设置不生效
-- 建议使用新接口 setBacklightParams

3.10 获取背光 PWM 最大值和最小值

接口名称：

void getBacklightPWM(int screen, ResultCallback<ResultInfo<String>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	回调 val：字符串 -- 前面是最小值 -- 后面是最大值 例：“10,100”，表示最小值是 10，最大值是 100.

代码示例：

```
SdkApi.getInstance().Display().getBacklightPWM(ApiTool.Screen.ALL
,
    new ResultCallback<ResultInfo<String>>() {
        @Override
        public void onMsg(String val) {

        }
})
```

备注：
-- 建议使用新接口 getBacklightParams

3.11 设置屏幕分辨率

接口名称：
boolean setResolution(int screen, String resolution)

参数：

参数名	类型	说明
screen	int	显示接口 ID
resolution	String	分辨率格式：宽 x 高 注意：screen 为 HDMI 时，需要加“@帧率” 例如： lvds：“1920x1980” hdmi：“3840x2160@25”

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setResolution(
ApiTool.Screen.LVDS, "1920x1080")
```

备注：
-- 重启生效

3.12 获取屏幕分辨率

接口名称：

void getResolution(int screen, ResultCallback<ResultInfo<String>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	回调 val：分辨率 格式："1920x1080@60"

代码示例：

```
    SdkApi.getInstance().Display().getResolution(ApiTool.Screen.ALL,
        new ResultCallback<ResultInfo<String>>() {
            @Override
            public void onMsg(String val) {

            }
    })
```

备注：

-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.13 设置 lvds 模式

接口名称：

boolean setLvdsMode(int screen, int mode)

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
mode	int	0: vesa 1: jeida

返回值：

参数名	类型	说明
返回值	boolean	true: 成功 false: 失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setLvdsMode(
ApiTool.Screen.ALL, 0)
```

备注：
-- 无

3.14 获取 lvds 模式

接口名称：

void getLvdsMode(int screen, ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	回调 val： 0：vesa 1：jeida

代码示例：

```
SdkApi.getInstance().Display().getLvdsMode(ApiTool.Screen.ALL,
    new ResultCallback<ResultInfo<Integer>>() {
        @Override
        public void onMsg(Integer val) {

        }
})
```

备注：

-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.15 设置 lvds Even-ODD 交换

接口名称：

boolean setLvdsSwap(int screen, int swap)

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
swap	int	0: no swap 1: swap

返回值：

参数名	类型	说明
返回值	boolean	true: 成功 false: 失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setLvdsMode(
ApiTool.Screen.ALL, 0)
```

备注：
-- 无

3.16 获取 lvds Even-ODD 交换

接口名称：

void getLvdsSwap(int screen, ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	回调 val： 0：no swap 1：swap

代码示例：

```
SdkApi.getInstance().Display().getLvdsSwap(ApiTool.Screen.ALL,
    new ResultCallback<ResultInfo<Integer>>() {
        @Override
        public void onMsg(Integer val) {

        }
})
```

备注：
-- 如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.17 设置 lvds 通道

接口名称：
`boolean setLvdsChannel(int screen, int ch)`

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
ch	int	0：single 1：dual

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display().setLvdsChannel(
ApiTool.Screen.ALL, 0)
```

备注：
-- 无

3.18 获取 lvds 通道

接口名称：

void getLvdsChannel(int screen, ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID 支持 -- LVDS -- LVDS2 -- ALL(只更新 LVDS 接口) 其它接口不支持
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	回调 val： 0：single 1：dual

代码示例：

```
SdkApi.getInstance().Display().getLvdsChannel(ApiTool.Screen.ALL,
    new ResultCallback<ResultInfo<Integer>>() {
        @Override
        public void onMsg(Integer val) {

        }
});
```

备注：
如果主板有双路输出，screen 填写 all 时，返回的是当前主屏的状态

3.19 获取支持的分辨率列表

接口名称：
`String getSupportedResolutionsList(int screen)`

参数：

参数名	类型	说明
screen	int	显示接口 ID

返回值：

参数名	类型	说明
回调返回参数 val	String	支持的分辨率列表字符串 如： “1920x1080@60,1366x768@60” 如支持列表为空，则返回“none”

代码示例：

```
String resolutionList =
SdkApi.getInstance().Display().getSupportedResolutionsList(ApiToo
l.Screen.LVDS)


-- 示例：1920x1080@60,1366x768@60
```

备注：
-- 当 ID 为 ALL 时，只返回 LVDS 接口数据

3.20 设置自定义分辨率 Timing

接口名称：
`boolean setCustomTiming(CustomTiming timing)`

参数：

参数名	类型	说明
timing	CustomTiming	自定义分辨率 Timing

结构说明

```
Class CustomTiming{
  int type;  //ApiTool.TimingType 内的屏索引
  int d_clk;
  int h_active;
  int v_active;
  int h_bp;
  int h_fp;
  int h_sync;
  int v_bp;
  int v_fp;
  int v_sync;
}

-- type 字段值定义
  >> 0: LVDS
  >> 1: OTHER2LVDS
  >> 2: MIPI
  >> 3: EDP
  >> 4: LVDS2
```

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display.setCustomTiming(new
CustomTiming(...));
```

备注：
-- 导入后需重启生效

3.21 设置自定义 mipi 指令

接口名称：
`boolean setCustomMipiCmds(int[] cmdsBuf, int cmdsLen)`

参数：

参数名	类型	说明
cmdsBuf	int[]	自定义 mipi 指令缓存
cmdsLen	int	数据长度

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display.setCustomMipiCmds(
cmdsBuf, cmdsLen);
```

备注：
-- 导入后需重启生效

3.22 设置背光参数

接口名称：
`int setBacklightParams(int screen, int type, int typeValue)`

参数：

参数名	类型	说明
screen	int	显示接口 ID
type	int	类型： ApiTool.BacklightType
typeValue	int	type: -- 不同 type 对应的 value 说明 type_pwm_dc: -- 0: dc -- 1: pwm type_inverse: -- 0: no inverse -- 1: inverse type_max: -- 0-100，建议>min type_min: -- 0-100，建议<max type_pwm_freq: -- 100-100000 -- 单位：Hz

返回值：

参数名	类型	说明
返回值	int	0：成功 -1：失败

代码示例：

```
int ret = SdkApi.getInstance().Display.setBacklightParams(
screen, type, typeValue);
```

备注：
-- 无

3.23 获取背光参数

接口名称：

int getBacklightParams(int screen, int type,ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
screen	int	显示接口 ID
type	int	类型：ApiTool.BacklightType
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	int	回调 val： type： -- 不同 type 对应的 value 说明 type_pwm_dc: -- 0: dc -- 1: pwm type_inverse: -- 0: no inverse -- 1: inverse type_max: -- 0-100，建议>min type_min: -- 0-100，建议<max type_pwm_freq: -- 100-100000 -- 单位：Hz

代码示例：

```
    SdkApi.getInstance().Display.getBacklightParams(
        screen, type, new ResultCallback<ResultInfo<Integer>>() {
            @Override
            public void onMsg(Integer val) {

            }
    });
```

备注：
-- 无

3.24 设置画面显示大小

-- 此接口适用于条形屏/切割屏

接口名称：
`boolean setScreenGraphicDisplay(int scale, int x_pos, int y_pos, int width, int height)`

参数：

参数名	类型	说明
scale	int	缩放开关： 0：关闭 1：打开 默认值：打开
x_pos	int	相对物理左上角的横向偏移
y_pos	int	相对物理左上角的纵向偏移
width	int	显示画面宽度
height	int	显示画面高度

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret = SdkApi.getInstance().Display.
setScreenGraphicDisplay (scale, x, y, width, height);
```

备注：
-- 设置后重启生效

3.25 获取画面显示位置

-- 此接口适用于条形屏/切割屏

接口名称：
void getScreenGraphicDisplay(ResultCallback<ResultInfo<String>> cb)

参数：

参数名	类型	说明
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	回调 val 格式：scale,0,0,1920,1080

代码示例：

```
    SdkApi.getInstance().Display().getScreenGraphicDisplay (
        new ResultCallback<ResultInfo<String>>() {
            @Override
            public void onMsg(String val) {

            }
    })
```

备注：
-- 无

4、 系统控制 API

-- 接口调用方式：SdkApi.getInstance().SystemCtrl().接口

4.1 设备重启

接口名称：
void deviceReboot()

参数：
-- 无

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().deviceReboot()
```

备注：
-- 无

4.2 截屏

接口名称：

void shotScreen(String path, int screen, ResultCallback<Boolean> cb)

参数：

参数名	类型	说明
path	String	截屏文件路径和名称
screen	int	主副屏 0：主屏 1：副屏 默认值：0
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调 val： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().SystemCtrl().shotScreen("/mnt/sdcard/1.png",
0, new ResultCallback<ResultInfo<Boolean>>() {
        @Override
        public void onMsg(Boolean val) {

        }
})
```

备注：
-- 截屏文件格式只能是 png，目前只支持主屏截屏

4.3 隐藏或者显示导航栏和状态栏

接口名称：
void setStatusBarDisplay(boolean display)

参数：

参数名	类型	说明
display	boolean	true：导航栏显示 false：导航栏隐藏

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().setStatusBarDisplay(false)
```

备注：
-- 旧接口，统一控制导航栏和状态栏
-- 此接口慢慢废弃

4.4 获取导航栏和状态栏状态

接口名称：
boolean getStatusBarDisplay()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：导航栏显示 false：导航栏隐藏

代码示例：

```
SdkApi.getInstance().SystemCtrl().getStatusBarDisplay()
```

备注：
-- 旧接口，统一获取导航栏和状态栏
-- 此接口慢慢废弃

4.5 隐藏或者显示状态栏

接口名称：
void setStatusBarDisplay2(boolean display)

参数：

参数名	类型	说明
display	boolean	true：状态栏显示 false：状态栏隐藏

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().setStatusBarDisplay2(false)
```

备注：
-- 无

4.6 获取状态栏状态

接口名称：
boolean getStatusBarDisplay2()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：状态栏显示 false：状态栏隐藏

代码示例：

```
boolean ret = SdkApi.getInstance().SystemCtrl().getStatusBarDisplay2()
```

备注：
-- 无

4.7 隐藏或者显示导航栏

接口名称：
void setNavigationBar(boolean display)

参数：

参数名	类型	说明
display	boolean	true：导航栏显示 false：导航栏隐藏

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().setNavigationBar(false)
```

备注：
-- 无

4.8 获取导航栏状态

接口名称：
boolean getNavigationBar()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：导航栏显示 false：导航栏隐藏

代码示例：

```
boolean ret =
SdkApi.getInstance().SystemCtrl().getNavigationBar()
```

备注：
-- 无

4.9 设备普通待机

接口名称：
void setDeviceShallowOnoff(boolean onoff)

参数：

参数名	类型	说明
onoff	boolean	true：开机 false：关机

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().setDeviceShallowOnoff(true)
```

备注：
-- 无

4.10 获取设备待机状态

接口名称：
boolean getDeviceShallowOnoff()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：开机 false：关机

代码示例：

```
boolean ret =
SdkApi.getInstance().SystemCtrl().getDeviceShallowOnoff()
```

备注：
-- 无

4.11 设备低功耗待机

接口名称：
void setDeviceDeepStandby()

参数：
-- 无

返回值：
-- 无

代码示例：

```
    SdkApi.getInstance().setDeviceDeepStandby()
```

备注：
-- 执行此接口后，设备将一直待机，下次开机触发条件
    --> 断上电
    --> 遥控或者按键开机

4.12 设备低功耗定时待机

接口名称：
`void setDeviceDeepStandby(int time)`

参数：

参数名	类型	说明
time	int	距离下次开机时间， 单位：分钟 范围：1-10080

返回值：
-- 无

代码示例：

```
//1 小时后开机
SdkApi.getInstance().SystemCtrl().setDeviceDeepStandby(1*60)
```

备注：
-- 无

4.13 设置风扇开关

接口名称：
`boolean setFanOnoff(boolean onoff)`

参数：

参数名	类型	说明
onoff	boolean	true：打开 false：关闭 默认值：关闭

返回值：
-- 无

代码示例：

```
boolean ret = SdkApi.getInstance().SystemCtrl().setFanOnoff(true)
```

备注：
-- 无

4.14 获取风扇开关状态

接口名称：
`boolean getFanOnoff()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：打开 false：关闭

代码示例：

```
boolean ret = SdkApi.getInstance().SystemCtrl().getFanOnoff()
```

备注：
-- 无

4.15 静默安装 App

接口名称：

void silentInstallApk(String path, boolean isAuto,

ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
path	String	待安装 APP 文件绝对路径
isAuto	boolean	true：安装完自动启动 App false：安装完不启动 App
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调 val： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().SystemCtrl().silentInstallApk("/mnt/sdcard/1
.apk", false, new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
})
```

备注：
-- 使用此接口前，需根据考虑实际业务场景，原因是
--> 当待安装 APP 文件存放在 U 盘内，APP 安装完毕后，再拔掉 U 盘，会导致 APP 被 kill，此种业务场景建议使用 4.16 接口

4.16 静默安装 App

接口名称：

void silentInstallApkForPackage(String path, String packageName, boolean isAuto, ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
path	String	待安装 APP 文件绝对路径
packageName	String	待安装 APP 包名，不可为空
isAuto	boolean	true：安装完自动启动 App false：安装完不启动 App
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调 val： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().SystemCtrl().silentInstallApkForPackage("/mn
t/sdcard/1.apk", "com.xx.xx", false, new
ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
})
```

备注：
-- 无

4.17 静默卸载 App

接口名称：

void silentUninstallApk(String packageName, ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
packageName	String	待卸载 APP 包名
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调 val： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().SystemCtrl().silentUninstallApk(
"com.signway.testdemo",
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
})
```

备注：
-- 无

4.18 获取触摸最后一次触摸时间

接口名称：
`long getTPLastTouchTime()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	long	最后一次触摸的系统时间，如系统一直没有触摸时，返回值为 0

代码示例：

```
long time =
SdkApi.getInstance().SystemCtrl().getTPLastTouchTime()
```

备注：
-- 无

4.19 格式化设备

接口名称：

void formatStorageDevice(int type, ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
type	int	格式化类型： -- 本地存储【0】 --> ApiTool.Format.Internal -- TF 卡【1】 --> ApiTool.Format.SDCard -- U 盘【2】 --> ApiTool.Format.Usb
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调 val： true：成功 false：失败

代码示例：

```
//格式化本地存储
SdkApi.getInstance().SystemCtrl().getTPLastTouchTime(0,
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
})
```

备注：
-- SDK 版本 >= 28，仅支持格式化本地存储

4.20 修改开机动画音量大小

接口名称：
long setBootVideoVolume(int volume)

参数：

参数名	类型	说明
volume	int	音量大小 范围：0-100

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
//将开机动画视频音量设置为 50%
Boolean ret =
SdkApi.getInstance().SystemCtrl().setBootVideoVolume(50)
```

备注：
-- 无

4.21 恢复出厂设置

接口名称：
`int recoverySystem(int type)`

参数：

参数名	类型	说明
type	int	0：恢复系统所有参数，包括本地存储的文件

返回值：

参数名	类型	说明
返回值	int	0：成功 -1：失败

代码示例：

```
int ret =
SdkApi.getInstance().SystemCtrl().recoverySystem(0)
```

备注：
-- 无

4.22 获取开机累计时长

接口名称：
`long getUpTime()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	long	累计时长

代码示例：

```
long uptime = SdkApi.getInstance().SystemCtrl().getUpTime();
```

备注：
-- 无

4.23 重置开机累计时长

接口名称：
`void resetUpTime(long msec)`

参数：

参数名	类型	说明
返回值	long	累计时长

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().resetUpTime(mSec);
```

备注：
-- 无

4.24 获取 NTP 地址

接口名称：
`String getNtpServerAddress()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	NTP 地址字符串

代码示例：

```
String address =
SdkApi.getInstance().SystemCtrl().getNtpServeSdress();
```

备注：
-- 无

4.25 重置 NTP 地址

接口名称：
`void setNtpServerAddress(String address)`

参数：

参数名	类型	说明
返回值	String	新的 NTP 地址

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemCtrl().setNtpServerAddress(address);
```

备注：
-- 无

5、 定时开关机 API

-- 接口调用方式：SdkApi.getInstance().TimerSwitch().接口

-- 功能描述：在时间段内开机，时间段外关机

示例：

开机时间 9:00，关机时间 10:00，设备工作时间为 9-10 点

开机时间 10:00，关机时间 9:00，设备工作时间为当天 10 点至第二天 9:00

开机时间=关机时间，无效设置，维持当前状态

5.1 设置定时开关

接口名称：
```
void setTimerSwitchOnoff(boolean onoff)
```

参数：

参数名	类型	说明
onoff	boolean	定时功能开关 默认值：关闭

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().TimerSwitch().setTimerSwitchOnoff(true)
```

备注：
-- 开机或者系统重启后 2 分钟之后开始计时，故开机 2 分钟内定时不生效

5.2 获取定时开关状态

接口名称：
`boolean getTimerSwitchOnoff()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：打开 false：关闭

代码示例：

```
boolean ret =
SdkApi.getInstance().TimerSwitch().getTimerSwitchOnoff()
```

备注：
-- 无

5.3 设置定时开机时间

接口名称：
`boolean setTimerOnTime(String onTime)`

参数：

参数名	类型	说明
onTime	String	定时开机时间 格式："HH:mm"

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
    boolean ret =
    SdkApi.getInstance().TimerSwitch().setTimerOnTime("12:01")
```

备注：
-- 无

5.4 清除定时开机时间

接口名称：
`void clearTimerOnTime()`

参数：
-- 无

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().TimerSwitch().clearTimerOnTime()
```

备注：
-- 默认时间：00:00

5.5 获取定时开机时间

接口名称：
`String getTimerOnTime()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	定时开机时间 格式："HH:mm"

代码示例：

```
String time = SdkApi.getInstance().TimerSwitch().getTimerOnTime()
```

备注：
-- 默认时间：00:00

5.6 设置定时关机时间

接口名称：
`boolean setTimerOffTime(String offTime)`

参数：

参数名	类型	说明
offTime	String	定时关机时间 格式："HH:mm"

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret =
SdkApi.getInstance().TimerSwitch().setTimerOffTime("12:01")
```

备注：
-- 无

5.7 清除定时关机时间

接口名称：
`void clearTimerOffTime()`

参数：
-- 无

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().TimerSwitch().clearTimerOffTime()
```

备注：
-- 默认时间：00:00

5.8 获取定时关机时间

接口名称：
`String getTimerOffTime()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	定时关机时间 格式："HH:mm"

代码示例：

```
String time =
SdkApi.getInstance().TimerSwitch().getTimerOffTime()
```

备注：
-- 默认时间：00:00

5.9 设置定时关机模式

接口名称：

void setTimerStandbyMode(boolean standby)

参数：

参数名	类型	说明
standby	boolean	false：普通待机 true：低功耗待机 默认值：false

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().TimerSwitch().setTimerStandbyMode(false)
```

备注：
-- 无

5.10 获取定时关机模式

接口名称：
`boolean getTimerStandbyMode()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	false：普通待机 true：低功耗待机

代码示例：

```
boolean ret =
SdkApi.getInstance().TimerSwitch().getTimerOffTime()
```

备注：
-- 无

6、 喂狗相关 API

-- 接口调用方式：SdkApi.getInstance().Watchdog().接口
-- 功能：看门狗复位，默认 sdk 喂狗，App 喂狗时，Sdk 启动 App 喂狗服务，监听 App 喂狗

6.1 设置喂狗主体

接口名称：
boolean setWatchdogOnoff(boolean onoff)

参数：

参数名	类型	说明
onoff	boolean	喂狗方是 APP 还是系统 true：APP false：系统 默认值：false

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
SdkApi.getInstance().Watchdog().setWatchdogOnoff(true)
```

备注：
-- 无

6.2 获取喂狗主体方

接口名称：
boolean getWatchdogOnoff()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：APP false：系统

代码示例：

```
SdkApi.getInstance().Watchdog().getWatchdogOnoff()
```

备注：
-- 无

6.3 设置看门狗超时时间

接口名称：
`boolean setWatchdogTimeout(int timeout)`

参数：

参数名	类型	说明
timeout	int	App 看门狗超时时间 默认：3 分钟 范围：30 秒-5 分钟 单位：秒

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
//设置喂狗超时时间为 60 秒
SdkApi.getInstance().Watchdog().setWatchdogTimeout(60)
```

备注：
-- 无

6.4 喂狗

接口名称：
`boolean feedWatchdog()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
SdkApi.getInstance().Watchdog().feedWatchdog()
```

备注：
-- 无

7、 系统时间相关 API

-- 接口调用方式：SdkApi.getInstance().SystemTime().接口

7.1 设置系统时间

接口名称：
void setSystemTime(long time)

参数：

参数名	类型	说明
time	long	距离 1970 年开始的时间差 单位：毫秒

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemTime().setSystemTime(60*1000*1000)
```

备注：
-- 无

7.2 获取 RTC 时间

接口名称：
`String getRtcTime()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	RTC 时间 时间格式："yyyy/MM/dd hh:mm:ss"

代码示例：

```
String time =
SdkApi.getInstance().TimerSwitch().getRtcTime()

返回值示例

2025/01/01 12:00:00
```

备注：
-- 无

7.3 设置自动校准时间

接口名称：
`void setAutoTimeOnoff(boolean onoff)`

参数：

参数名	类型	说明
onoff	boolean	true：NTP 打开 false：NTP 关闭

返回值：
-- 无

代码示例：

```
SdkApi.getInstance().SystemTime().setAutoTimeOnoff(true)
```

备注：
-- 无

7.4 获取自动校准时间开关状态

接口名称：
`boolean getAutoTimeOnoff()`

参数：
-- 无

返回值：

参数名	类型	说明
返回值	boolean	true：NTP 打开 false：NTP 关闭

代码示例：

```
SdkApi.getInstance().SystemTime().getAutoTimeOnoff()
```

备注：
-- 无

8、 硬件相关 API

-- 接口调用方式：SdkApi.getInstance().Hardware().接口

8.1 获取 GPIO 输入电平

接口名称：
int getGpioInput(int group, int num)

参数：

参数名	类型	说明
group	int	GPIO 所在组； 例：GPIO0_B3，group 值是 GPIO0 详见“**GPIO 值定义**”
num	int	GPIO 在组内的序号； 例：GPIO0_B3，num 值是 B3 详见“**GPIO 值定义**”

返回值：

参数名	类型	说明
返回值	boolean	1:高电平 0:低电平 -1：读取错误

代码示例：

```
//GPIO0_B3
int level = SdkApi.getInstance().Hardware().getGpioInput(
ApiTool.GpioPin.GPIO0, ApiTool.GpioPin.PB3);
```

备注：
-- 无

8.2 设置 GPIO 输出状态

接口名称：
`boolean setGpioOutput(int group, int num, boolean level)`

参数：

参数名	类型	说明
group	int	GPIO 所在组； 例：GPIO0_B3，group 值是 GPIO0 详见“**GPIO 值定义**”
num	int	GPIO 在组内的序号； 例：GPIO0_B3，num 值是 B3 详见“**GPIO 值定义**”
level	boolean	true：高电平 false：低电平

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
//GPIO0_B3
int level = SdkApi.getInstance().Hardware().setGpioOutput(
ApiTool.GpioPin.GPIO0, ApiTool.GpioPin.PB3, true);
```

备注：
-- 无

8.3 读取 flash 数据

接口名称：
`void flashRead(int type, int len, ResultCallback<ResultInfo<Integer>> cb)`

参数：

参数名	类型	说明
type	int	数据类型 -- HWID -- 授权信息 等等 详见 ApiTool.FlashRWType 类
len	int	需要读取的数据长度
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	回调 val1：读取的数据 val2：读取的实际长度

代码示例：

```
SdkApi.getInstance().Hardware().flashRead(
ApiTool.FlashRWType.HWID, 1,
new ResultCallback<ResultInfo<Interger>>() {
    @Override
    public void onMsg(ResultInfo<Integer> val) {
        byte[] array = val.getArrayByte();
        int len = val.getVal();
    }
});
```

备注：

-- 读取 type 的整个数据块
-- 数据最大长度：1024 字节；部分 type 只有几个字节

8.4 写 flash 数据

接口名称：

```
void flashWrite(int type, int len, int[] buffer, ResultCallback<ResultInfo<Boolean>> cb)
```

参数：

参数名	类型	说明
type	int	数据类型 -- HWID -- 授权信息 等等 详见 ApiTool.FlashRWType 类
len	int	写入数据长度
buffer	int[]	写入数据 buffer
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	true：成功 false：失败

代码示例：

```
    SdkApi.getInstance().Hardware().flashWrite(
    ApiTool.FlashRWType.APP_INFO, 1, buffer
    new ResultCallback<ResultInfo<Boolean>>() {
        @Override
        public void onMsg(Boolean val) {

        }
    });
```

备注：
-- 无

8.5 主板运行指示灯

接口名称：
`void setRunningLedOnoff(boolean onoff, ResultCallback<ResultInfo<Boolean>> cb)`

参数：

参数名	类型	说明
onoff	boolean	true：打开 false：关闭
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调获取设置结果： true：成功 false：失败

代码示例：

```
//打开运行指示灯
SdkApi.getInstance().Hardware().setRunningLedOnoff(true,
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
});
```

备注：

-- 不同主板配置有差异，存在
    --> 主板无指示灯
    --> 指示灯由硬件电路决定
-- 默认值：打开

8.6 获取主板运行指示灯

接口名称：

```
void getRunningLedOnoff(ResultCallback<ResultInfo<Boolean>> cb)
```

参数：

参数名	类型	说明
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	String	回调获取设置结果： true：打开 false：关闭

代码示例：

```
    SdkApi.getInstance().Hardware().getRunningLedOnoff(
    new ResultCallback<ResultInfo<Boolean>>() {
        @Override
        public void onMsg(Boolean val) {

        }
    });
```

备注：

-- 不同主板配置有差异，存在
    --> 主板无指示灯
    --> 指示灯由硬件电路决定
-- 默认值：打开

8.7 设置按键板红色指示灯开关

接口名称：

`void setKeyBoardLedROnoff(boolean onoff, ResultCallback<ResultInfo<Boolean>> cb)`

参数：

参数名	类型	说明
onoff	boolean	true：打开 false：关闭
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调获取设置结果： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().Hardware().setKeyBoardLedROnoff(true,
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
});
```

备注：
-- 以主板为准，默认按照 LED 共阴设计控制

8.8 获取按键板红色指示灯开关

接口名称：
`void getKeyBoardLedROnoff(ResultCallback<ResultInfo<Boolean>> cb)`

参数：

参数名	类型	说明
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调获取状态 true：打开 false：关闭

代码示例：

```
SdkApi.getInstance().Hardware().getKeyBoardLedROnoff(
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
});
```

备注：

-- 以主板为准，默认按照 LED 共阴设计控制

8.9 设置按键板绿色指示灯开关

接口名称：

void setKeyBoardLedGOnoff(boolean onoff,ResultCallback<ResultInfo<Boolean>> cb)

参数：

参数名	类型	说明
onoff	boolean	true：打开 false：关闭
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调获取设置结果： true：成功 false：失败

代码示例：

```
SdkApi.getInstance().Hardware().setKeyBoardLedGOnoff(true,
new ResultCallback<ResultInfo<Boolean>>() {
    @Override
    public void onMsg(Boolean val) {

    }
});
```

备注：
-- 以主板为准，默认按照 LED 共阴设计控制

8.10 获取按键板绿色指示灯开关

接口名称：
`void getKeyBoardLedGOnoff(ResultCallback<ResultInfo<Boolean>> cb)`

参数：

参数名	类型	说明
cb	class	ResultCallback 类

返回值：

参数名	类型	说明
回调返回参数 val	boolean	回调获取状态 true：打开 false：关闭

代码示例：

```
    SdkApi.getInstance().Hardware().getKeyBoardLedGOnoff(
    new ResultCallback<ResultInfo<Boolean>>() {
        @Override
        public void onMsg(Boolean val) {

        }
    });
```

备注：
-- 以主板为准，默认按照 LED 共阴设计控制

~~8.11 设置 USB 端口电源开关~~

接口名称：
`boolean setUsbPortPowerOnoff(int usbPort, boolean onoff)`

参数：

参数名	类型	说明
usbPort	int	usb 端口索引号
onoff	boolean	true：打开 false：关闭

返回值：
--无
代码示例：

```
SdkApi.getInstance().Hardware().setUsbPortPowerOnoff(port, true);
```

备注：
-- 无此接口
-- 以主板为准，usb 端口索引号不同

~~8.12 获取 USB 端口电源开关~~

接口名称：
`boolean getUsbPortPowerOnoff(int usbPort)`

参数：

参数名	类型	说明
usbPort	int	usb 端口索引号

返回值：

参数名	类型	说明
返回值	boolean	true：打开 false：关闭

代码示例：

```
SdkApi.getInstance().Hardware().getUsbPortPowerOnoff(port);
```

备注：
-- 无此接口
-- 以主板为准，usb 端口索引号不同

8.13 设置喇叭功率参数

接口名称：
`boolean setSpeakerConfigsSave(boolean status, int power, int resistance)`

参数：

参数名	类型	说明
status	boolean	喇叭参数保存状态 true：使能 false：关闭 默认值：以板卡默认配置为准
power	int	喇叭功率：1-10w
resistance	int	喇叭阻值：2、4、6、8Ω

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
//配置喇叭为 2 瓦 4 欧
boolean ret =
SdkApi.getInstance().Hardware().setSpeakerConfigsSave(status, 2,
4);
```

备注：
--无

8.14 获取喇叭功率参数

接口名称：

void getSpeakerConfigs(ResultCallback<ResultInfo<Integer>> cb)

参数：

参数名	类型	说明
cb	class	获取喇叭功率回调

返回值：

参数名	类型	说明
回调返回参数 val	int	回调获取数组： 第一个数据是 status 第二个数据是喇叭功率 第三个数据是喇叭阻值

代码示例：

```
SdkApi.getInstance().Hardware().getSpeakerConfigs(
new ResultCallback<ResultInfo<Integer>>() {
    ...
    byte[] array = val.getArrayByte();
    array[0] //status
    array[1] //功率
    array[2] //阻值
});
```

备注：
--无

8.15 打开串口设备 1

-- 只支持按照 8N1 方式打开串口

接口名称：
`int openSerial(String serialName, int baud)`

参数：

参数名	类型	说明
serialName	String	串口设备名称 例：“/dev/ttyS0”
baud	int	串口波特率

返回值：

参数名	类型	说明
返回值	int	串口设备句柄

代码示例：

```
int serialFd =
SdkApi.getInstance().Hardware().openSerial("/dev/ttyS0", 115200);
```

备注：
-- 无

8.16 打开串口设备 2

-- 通用方法

接口名称：
`int openSerial(String serialName, int baud, int dataBits, int parity, int stopBits)`

参数：

参数名	类型	说明
serialName	String	串口设备名称 例：“/dev/ttyS0”
baud	int	串口波特率
dataBits	int	数据位 值：5、6、7、8
parity	int	奇偶校验 0：NONE 1：ODD 2：EVEN
stopBits	int	停止位 值：1、2

返回值：

参数名	类型	说明
返回值	int	串口设备句柄

代码示例：

```
int serialFd = SdkApi.getInstance().Hardware().openSerial("/dev/ttyS0",
115200,  8, 0, 1);
```

备注：
-- 无

8.17 关闭串口设备

接口名称：
`int closeSerial(int serial)`

参数：

参数名	类型	说明
serial	int	串口设备句柄

返回值：

参数名	类型	说明
返回值	int	0：关闭成功 其他值：关闭失败

代码示例：

```
int ret = SdkApi.getInstance().Hardware().closeSerial(serial);
```

备注：
-- 无

8.18 读串口数据

接口名称：
`int readSerial(int serial, byte[] buf, int len)`

参数：

参数名	类型	说明
serial	int	对应串口的句柄
buf	byte[]	保存读数据的缓存
len	int	需要读取的数据长度

返回值：

参数名	类型	说明
返回值	int	实际读取的数据长度

代码示例：

```
int readLen = SdkApi.getInstance().Hardware().readSerial(serial,
buf, len);
```

备注：
-- 无

8.19 写串口数据

接口名称：

int writeSerial(int serial, byte[] buf, int len)

参数：

参数名	类型	说明
serial	int	对应串口的句柄
buf	byte[]	写入数据的缓存
len	int	需要写入的数据长度

返回值：

参数名	类型	说明
返回值	int	实际写入的数据长度

代码示例：

```
    int writeLen =
    SdkApi.getInstance().Hardware().writeSerial(serial, buf, len);
```

备注：
-- 无

8.20 配置并打开 can 总线

接口名称：

int canBusConfigUp(int bitrate)

参数：

参数名	类型	说明
bitrate	int	can 总线速率

返回值：

参数名	类型	说明
返回值	int	0：成功 其他值：失败

代码示例：

```
int ret = SdkApi.getInstance().Hardware().canBusConfigUp(250000);
```

备注：
-- 无

8.21 打开 can 设备

接口名称：

int canOpen(int canId)

参数：

参数名	类型	说明
canId	int	can 总线 id

返回值：

参数名	类型	说明
返回值	int	can 设备句柄

代码示例：

```
int ret = SdkApi.getInstance().Hardware().canOpen(0);
```

备注：
-- id 默认为 0

8.22 发送 can 帧数据

接口名称：

int canSend(int can, int frameFlag, int frameId, byte[] canData, int len)

参数：

参数名	类型	说明
can	int	can 设备句柄
frameFlag	int	帧类型 如：CAN_EFF_FLAG
frameId	int	帧标识符
canData	byte[]	帧数据
len	int	帧数据长度 最大 8 个字节

返回值：

参数名	类型	说明
返回值	int	can 设备句柄

代码示例：

```
int ret =
SdkApi.getInstance().Hardware().canSend(can,0,0x123,0x11,1);
```

备注：
--无

8.23 发送 can 帧数据

接口名称：

int canRecv(int can, int[] canInfo)

参数：

参数名	类型	说明
can	int	can 设备句柄
canInfo	int[]	获取的数据 canInfo[0]为 can_id canInfo[1]为 can_dlc canInfo[2] 为 can_data 起始数据

返回值：

参数名	类型	说明
返回值	int	整个 canInfo 实际数据长度

代码示例：

```
    int[] canInfo = new int[16];
    int ret = SdkApi.getInstance().Hardware().canRecv(can, canInfo);
```

备注：
--无

8.24 关闭 can 设备

接口名称：

int canClose(int can)

参数：

参数名	类型	说明
can	int	can 设备句柄

返回值：

参数名	类型	说明
返回值	int	0：成功 其他值：失败

代码示例：

```
int ret = SdkApi.getInstance().Hardware().canClose(can);
```

备注：
--无

8.25 获取遥控器状态

接口名称：

int getRcuState()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	int	0：打开 1：全部关闭 2：除了电源按键全部关闭

代码示例：

```
int ret = SdkApi.getInstance().Hardware().getRcuState();
```

备注：
--无

8.26 设置遥控器状态

接口名称：

void setRcuState(int state)

参数：

参数名	类型	说明
can	int	遥控器状态值 0：打开 1：全部关闭 2：除了电源按键全部关闭

返回值：
-- 无

代码示例：

```
int rcuState = 0;

SdkApi.getInstance().Hardware().setRcuState(rcuState);
```

备注：
--无

8.27 获取串口电源开关

接口名称：

int getSerialPortOnOff(String serialName)

参数：

参数名	类型	说明
serialName	String	串口设备名称 例：“/dev/ttyS0”

返回值：

参数名	类型	说明
返回值	int	

代码示例：

```
int ret =
SdkApi.getInstance().Hardware().getSerialPortOnOff("/dev/ttyS0");
```

备注：
--无

8.28 设置串口电源开关

接口名称：

boolean setSerialPortOnOff(String serialName, Boolean onOff);

参数：

参数名	类型	说明
serialName	String	串口设备名称 例：“/dev/ttyS0”
onOff	boolean	true:打开 false：关闭

返回值：

参数名	类型	说明
返回值	boolean	true：成功 false：失败

代码示例：

```
boolean ret =
SdkApi.getInstance().Hardware().setSerialPortOnOff("/dev/ttyS0",
true);
```

备注：
--无

9、 其他相关 API

-- 接口调用方式：SdkApi.getInstance().Other().接口

9.1 获取 Sim 卡 ICCID 号

接口名称：
String getSimICCID()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	val: iccid

代码示例：

```
String iccid = SdkApi.getInstance().Other().getSimICCID();
```

备注：
-- 板卡需正确接入 4G 模块、插入 SIM 卡

9.2 获取 IMEI 号

接口名称：
String getModemIMEI()

参数：
-- 无

返回值：

参数名	类型	说明
返回值	String	val: imei

代码示例：

```
String imei = SdkApi.getInstance().Other().getModemIMEI();
```

备注：
-- 板卡需正确接入 4G 模块、插入 SIM 卡

9.3 执行 shell 命令

接口名称：

void execShellCommand(String cmd)

参数：

参数名	类型	说明
cmd	String	shell 命令 例：input keyevent 26

返回值：
--无

代码示例：

```
int ret =
SdkApi.getInstance().Other().execShellCommand("input keyevent
26");
```

备注：
--无

9.4 读取 emmc 寿命

接口名称：
`String getEmmcLeftTime()`

参数：
--无

返回值：

参数名	类型	说明
返回值	String	val: EMMC 剩余寿命值 例：010101

代码示例：

```
String leftTime = SdkApi.getInstance().Other().getEmmcLeftTime();
```

备注：
--无

10、 Scale 相关 API

-- 以实际主板为准，部分主板没有 scale 芯片，不支持该类接口

10.1 注册 scale 监听 listener

接口名称：
`int registerScaleListener(int appId, ScaleListener listener)`

参数：

参数名	类型	说明
appId	int	app 类型 Id 定义在 ApiTool.ScaleAppId
listener	ScaleListener	监听 listener

返回值：

参数名	类型	说明
返回值	String	是否注册成功 ApiTool.API_RES_*

代码示例：

```
int registerScaleListener(ApiTool.ScaleAppId.ID_APP_CUSTOM, new
ScaleListener() { });
```

备注：
-- 无

10.2 注销 scale 监听 listener

接口名称：
`int unregisterScaleListener(int appId, ScaleListener listener)`

参数：

参数名	类型	说明
appId	int	app 类型 Id，定义在 ApiTool.ScaleAppId
listener	ScaleListener	监听 listener

返回值：

参数名	类型	说明
返回值	String	是否注销成功 ApiTool.API_RES_*

代码示例：

```
int unregisterScaleListener(ApiTool.ScaleAppId.ID_APP_CUSTOM, new
ScaleListener() { });
```

备注：
-- 无

10.3 设置 scale 执行事件

接口名称：
int setScaleEvent(int appId, int actionId, String strParam, int[] dataArray, int arrayLen)

参数：

参数名	类型	说明
appId	int	app 类型 Id 定义在 ApiTool.ScaleAppId
actionId	int	事件 Id 定义在 ApiTool.ScaleEvent
strParam	String	参数，可以为 null
dataArray	int[]	参数，可以为 null 为 null 时，arrayLen 需为 0
arrayLen	int	dataArray 数据长度

返回值：

参数名	类型	说明
返回值	String	是否设置成功 事件结果在 listener 通知

代码示例：

```
int setScaleEvent(ApiTool.ScaleAppId.ID_APP_CUSTOM,
ApiTool.ScaleEvent.QUERY_VERSION);
```

备注：
-- 无

10.4 actionId 命令集

命令名	命令码	说明	备注
SYSTEM_EVENT_GET_SCALE_VERSION	0x8001	获取 scaler 版本	设置参数【dataArray[0]】: -- int[] dataArray = {1}; 返回值【dataArray】: -- 使用示例 new String(dataArray, 0, dataLen)
SYSTEM_EVENT_SET_SYSTEM_SHUTDOWN	0x8002	安全关机	设置参数【dataArray[0]】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
~~***SYSTEM_EVENT_SET_SYSTEM_POWER_ONOFF***~~	~~0x8003~~		
SYSTEM_EVENT_SET_SYSTEM_REBOOT	0x8004	主机重启	设置参数【dataArray[0]】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
SYSTEM_EVENT_GET_SCALE_TYPE	0x8005	获取产品定义	设置参数【dataArray[0]】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: 0x01：L 0x02：V 0x03：L2 0x04：VL 0x05：HL 0x06：S 0x07：SL 0x08：D

			0x09：DL 0x0a：F 0x0b：FL 0x0c：HV 0x0d：HL1
~~SCALE_EVENT_GET_PANEL_PARAM~~	~~0x3001~~	~~获取屏参~~	设置参数【dataArray】：~~int[] dataArray = {1};~~ ~~返回值【dataArray】：~~ ~~-- 0x01：代表屏幕类型 1~~ -- 0x02：代表屏幕类型 1
~~SCALE_EVENT_SET_PANEL_PARAM~~	0x3002	~~设置屏参~~	
SCALE_EVENT_GET_SCREEN_ONOFF	0x3003	获取屏幕开关状态	设置参数【dataArray[0]】： -- int[] dataArray = {1}; 返回值【dataArray[0]】： -- 0x01：屏幕状态为开 -- 0x00：屏幕状态为关
SCALE_EVENT_SET_SCREEN_ONOFF	0x3004	设置屏幕开关状态	设置参数【dataArray[0]】： -- 0x01：屏幕状态为开 -- 0x00：屏幕状态为关 返回值【dataArray[0]】： -- 0x10：代表成功 -- 0x11：代表失败
SCALE_EVENT_GET_INPUT_SOURCE	0x3005	获取信号源	设置参数【dataArray[0]】： -- int[] dataArray = {1}; 返回值【dataArray[0]】： -- 0x01：Android -- 0x02：Hdmi
SCALE_EVENT_SET_INPUT_SOURCE	0x3006	设置信号源	设置参数【dataArray[0]】： -- 0x01：Android -- 0x02：Hdmi 返回值【dataArray[0]】： -- 0x10：代表成功 -- 0x11：代表失败

SCALE_EVENT_GET_BACKLIGHT_ONOFF	0x3007	获取背光开关	设置参数【dataArray[0]】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x01：背光状态为开 -- 0x00：背光状态为关
SCALE_EVENT_SET_BACKLIGHT_ONOFF	0x3008	设置背光开关	设置参数【dataArray[0]】: -- 0x01：背光状态为开 -- 0x00：背光状态为关 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
SCALE_EVENT_GET_OSD_ORIENTATION	0x3009	获取 OSD 方向	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x01：0° -- 0x02：90° -- 0x03：180° -- 0x04：270°
SCALE_EVENT_SET_OSD_ORIENTATION	0x300A	设置 OSD 方向	设置参数【dataArray[0]】: -- 0x01：0° -- 0x02：90° -- 0x03：180° -- 0x04：270° 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
SCALE_EVENT_REPORT_INPUT_SOURCE	0x300B	上报信号通道	给所有注册的应用回调 返回值【dataArray[0]】: -- 0x01：Android -- 0x02：Hdmi
~~***SYSTEM_EVENT_SET_FUNCTION_KEYS***~~	~~0x300C~~	~~发送功能键~~	
SCALE_EVENT_REPORT_SCREEN_TYPE	0x300D	上报产品类型	给所有注册的应用回调 返回值【dataArray[0]】: 0x01：L

			0x02：V 0x03：L2 0x04：VL 0x05：HL 0x06：S 0x07：SL 0x08：D 0x09：DL 0x0a：F 0x0b：FL 0x0c：HV 0x0d：HL1
SCALE_EVENT_REPORT_SCREEN_VOLUME	0x300E	上报音量	给所有注册的应用回调 返回值【dataArray[0]】
~~SCALE_EVENT_GET_BACKLIGHT~~	~~0x3010~~	~~获取背光大小~~	
~~SCALE_EVENT_SET_BACKLIGHT~~	~~0x3011~~	~~设置背光大小~~	
~~SCALE_EVENT_QUERY_MUTE_STATUS~~	~~0x3012~~	~~获取 MUTE(静音)状态~~ ~~(硬件 MUTE)~~	
~~SCALE_EVENT_SET_MUTE_STATUS~~	~~0x3013~~	~~设置 MUTE(静音)状态~~ ~~(硬件 MUTE)~~	
~~SCALE_EVENT_QUERY_OSD_LANGUAGE~~	~~0x3014~~	~~获取语言~~	
~~SCALE_EVENT_SET_OSD_LANGUAGE~~	~~0x3015~~	~~设置语言~~	
AUDIO_EVENT_GET_VOLUME	0x3101	获取音量大小	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】
AUDIO_EVENT_SET_VOLUME	0x3102	设置音量大小	设置参数【dataArray】: -- 0 - 100 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
AUDIO_EVENT_GET_MUTE_STATUS	0x3103	获取 MUTE(静音)状态 (软件 MUTE)	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】:

			-- 0x00：代表静音状态为关（未静音） -- 0x01：代表静音状态为开（已静音）
AUDIO_EVENT_SET_MUTE_STATUS	0x3104	设置 MUTE(静音)状态 (软件 MUTE)	设置参数【dataArray】: -- 0x00：代表静音状态为关（未静音） -- 0x01：代表静音状态为开（已静音） 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
AUDIO_EVENT_GET_HORN_OUTPUT_POWER	0x3107	获取喇叭输出功率	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray】: -- dataArray[0]：代表喇叭阻值-欧数，一般为 4/8 -- dataArray[1]：喇叭功率-瓦数，一般为 1~10
AUDIO_EVENT_SET_HORN_OUTPUT_POWER	0x3108	设置喇叭输出功率	设置参数【dataArray】: -- dataArray[0]：代表喇叭阻值-欧数，一般为 4/8 -- dataArray[1]：喇叭功率-瓦数，一般为 1~10 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
PICTURE_EVENT_GET_BACKLIGHT_LEVEL	0x3201	获取图片亮度	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】:
PICTURE_EVENT_SET_BACKLIGHT_LEVEL	0x3202	设置图片亮度	设置参数【dataArray】: 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败

PICTURE_EVENT_GET_CONTRAST	0x3203	获取对比度	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0 - 100
PICTURE_EVENT_SET_CONTRAST	0x3204	设置对比度	
PICTURE_EVENT_GET_SHARPNESS	0x3205	空	
PICTURE_EVENT_SET_SHARPNESS	0x3206	空	
PICTURE_EVENT_GET_COLOR_TEMP	0x3207	获取色温	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x01：warm -- 0x02：standard -- 0x03：cool -- 0x04：user
PICTURE_EVENT_SET_COLOR_TEMP	0x3208	设置色温	设置参数【dataArray】: -- 0x01：warm -- 0x02：standard -- 0x03：cool -- 0x04：user 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
PICTURE_EVENT_GET_R_GAIN	0x3209	获取 R Gain	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0 - 255
PICTURE_EVENT_SET_R_GAIN	0x320A	设置 R Gain	设置参数【dataArray】: -- 0 - 255 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
PICTURE_EVENT_GET_G_GAIN	0x320B	获取 G Gain	设置参数【dataArray】: -- int[] dataArray = {1};

			返回值【dataArray[0]】: -- 0 - 255
PICTURE_EVENT_SET_G_GAIN	0x320C	设置 G Gain	设置参数【dataArray】: -- 0 - 255 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
PICTURE_EVENT_GET_B_GAIN	0x320D	获取 B Gain	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0 - 255
PICTURE_EVENT_SET_B_GAIN	0x320E	设置 B Gain	设置参数【dataArray】: -- 0 - 255 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
SCALE_EVENT_SET_LOGO_STYLE	0x3304	设置 logo 样式	设置参数【dataArray】: -- 0x00：Android -- 0x02：Goodview -- 0x03：Videri 返回值【dataArray[0]】: -- 0x10：代表成功 -- 0x11：代表失败
SCALE_EVENT_GET_LOGO_STYLE	0x3305	获取当前 logo 样式	设置参数【dataArray】: -- int[] dataArray = {1}; 返回值【dataArray[0]】: -- 0x00：Android -- 0x02：Goodview -- 0x03：Videri

10.4.1 上报屏参索引

执行代码示例

```
int setScaleEvent(ApiTool.ScaleAppId.ID_APP_CUSTOM,
ApiTool.ScaleEvent.SCALE_EVENT_REPORT_SCREEN_TYPE);
```

回调代码示例

```
ScaleListener scaleListener = new ScaleListener() {
    @Override
    public void onNotify(int actionId, byte[] dataArray, int
dataLen) {
          switch (actionId) {
              case SCALE_EVENT_REPORT_SCREEN_TYPE:
                    //todo your things
                    break;
         }
    }
};

方法说明：
-- 重写 ScaleListener 回调接口，调用 registerScaleListener
返回参数说明：
-- actionId : SCALE_EVENT_REPORT_SCREEN_TYPE
-- dataArray : 返回数据的数组[长度为 1byte]，具体值待定义
```

10.4.2 上报屏幕亮度

执行代码示例

```
int setScaleEvent(ApiTool.ScaleAppId.ID_APP_CUSTOM,
ApiTool.ScaleEvent.SCALE_EVENT_REPORT_SCREEN_VOLUME);
```

回调代码示例

```
ScaleListener scaleListener = new ScaleListener() {
    @Override
    public void onNotify(int actionId, byte[] dataArray, int
dataLen) {
          switch (actionId) {
              case SCALE_EVENT_REPORT_SCREEN_VOLUME:
                   //todo your things
                   break;
         }
    }
};
```

方法说明：

-- 重写 ScaleListener 回调接口，调用 registerScaleListener

返回参数说明：

-- actionId : SCALE_EVENT_REPORT_SCREEN_VOLUME

-- *dataArray* ：返回数据的数组[长度为 *1byte*]，值范围：*0-100*

11、 CommonEvent 相关 API

11.1 注册 Common 监听 listener

接口名称：

```
int registerCommonListener(int appId, ISdkCommonListener listener)
```

参数：

参数名	类型	说明
appId	int	app 类型 Id 定义在 ApiTool.CommonAppId
listener	ScaleListener	监听 listener

返回值：

参数名	类型	说明
返回值	String	是否注册成功 ApiTool.API_RES_*

代码示例：

```
    int registerCommonListener(ApiTool.CommonAppId.ID_APP_CUSTOM, new
    ISdkCommonListener() { });

    private ISdkCommonListener commonListener = new
    ISdkCommonListener.Stub() {
            @Override
            public void CommonListenerNotify(int actionId, String
    sVal, int[] iArray, int intArrayLen) throws RemoteException {
                if (actionId == ApiTool.CommonEvent.SWITCH_ETH) {
                    //设置以太网开关结果
                }
            }

            @Override
            public IBinder asBinder() {
                return super.asBinder();
            }
    };
```

备注：
--无

11.2 注销 Common 监听 listener

接口名称：
`int unregisterCommonListener(int appId, ISdkCommonListener listener)`

参数：

参数名	类型	说明
appId	int	app 类型 Id 定义在 ApiTool.CommonAppId
listener	listener	监听 listener

返回值：

参数名	类型	说明
返回值	String	是否注销成功 ApiTool.API_RES_*

代码示例：

```
int unregisterCommonListener(ApiTool.CommonAppId.ID_APP_CUSTOM,
listener);
```

备注：
-- 无

11.3 设置 Common 执行事件

接口名称：
int setCommonEvent(int appId, int actionId, String strParam, int[] dataArray, int arrayLen)

参数：

参数名	类型	说明
appId	int	app 类型 Id 定义在 ApiTool.CommonAppId
actionId	int	事件 Id 定义在 ApiTool.ScaleEvent
strParam	String	参数，可以为 null
dataArray	int[]	参数 如不用，请初始化为空数组，务必不要为 null
arrayLen	int	dataArray 数据长度

返回值：

参数名	类型	说明
返回值	String	是否设置成功，事件结果在 listener 通知，详情见 11.4 ISdkCommonListener 接口说明

代码示例：

```
int setCommonEvent(ApiTool.CommonAppId.ID_APP_CUSTOM,
ApiTool.CommonEvent.EVENT_GET_LN);
```

备注：
--无

11.4 ISdkCommonListener 接口说明

接口名称：
ISdkCommonListener

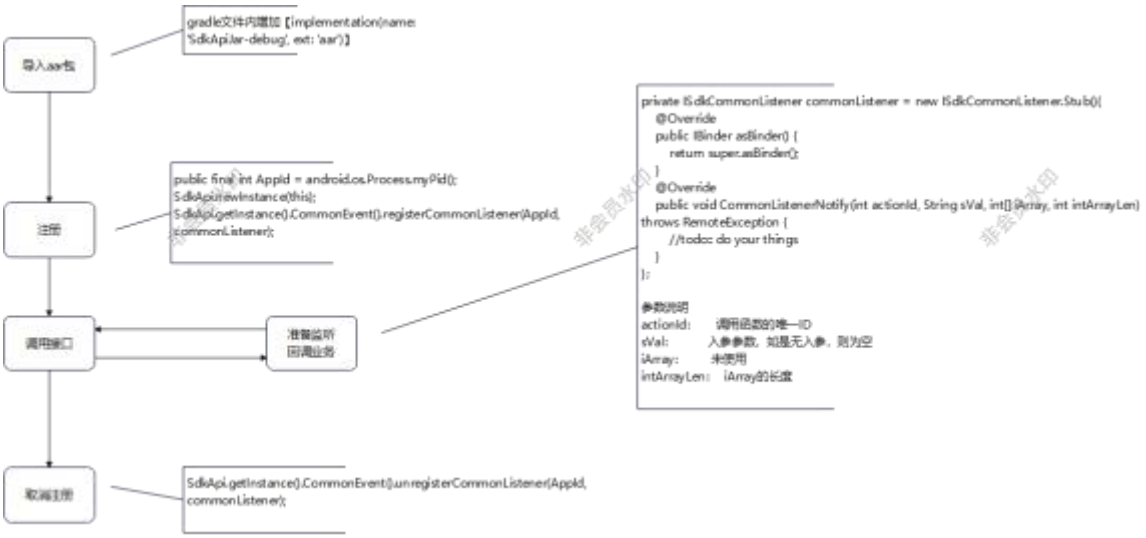
参数：

参数名	类型	说明
actionId	int	事件 Id 定义在 ApiTool.CommonEvent 与
sVal	String	接口返回参数，或者是接口执行结果
iArray	int[]	一般无特殊意义，备用返回数组，可能协助 intArrayLen 参数一起返回系统接口的运行结果
intArrayLen	int	iArray 数据长度，此时接收到的 length 长度实际是接口执行回调的成功或失败状态乃至于失败时错误码

返回值：
--无
代码示例：
--无

备注：
回调是 setCommonEvent 执行后触发，便于接收有返回值的接口

11.5 CommonEvent 使用流程



11.6 调用接口使用示例

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId, actionId, params, dataArray, dataArray.length());
```

参数说明：

-- AppId：调用端 app 的进程 ID

-- actionId：调用函数的唯一 ID

-- params ： 入参 ： 如果是多参数，则 params 表示 json 字符串

-- dataArray ： 备用参数，如不用，请初始化为空数组，务必不要为 null

dataArray.length()

11.7 actionId 命令集

命令名	命令码	说明	备注
EVENT_GET_LT_VERSION	0x1003	获取龙讯芯片版本	
SWITCH_WIFI	0x9000	WiFi 功能开关	
GET_WIFI_MAC	0x9001	获取 WiFi MAC	
SWITCH_BLUETOOTH	0x9010	蓝牙功能开关	
GET_BLUETOOTH_MAC	0x9011	获取蓝牙 MAC	
SWITCH_ETH	0x9020	以太网功能开关	
SWITCH_AP	0x9030	AP 功能开关	
GET_ETH_STATUS	0x9040	获取以太网状态	
SET_ETH_IP	0x9050	设置以太网 IP	
GET_ETH_INFO	0x9060	获取网络信息	
GET_AP_INFO	0x9070	获取 AP 信息	
SET_SCREEN_SATURATION	0x9080	设置屏幕饱和度	
GET_SCREEN_SATURATION	0x9081	获取屏幕饱和度	
SET_SCREEN_CONTRAST	0x9082	设置屏幕对比度	
GET_SCREEN_CONTRAST	0x9083	获取屏幕对比度	
SET_SCREEN_HUE	0x9084	设置屏幕色调	

GET_SCREEN_HUE	0x9085	获取屏幕色调	
SET_SCREEN_SIGNAL_BRIGHTNESS	0x9086	设置屏幕信号亮度	
GET_SCREEN_SIGNAL_BRIGHTNESS	0x9087	获取屏幕信号亮度	
UPDATE_SYSTEM_LOGO	0x9090	更新系统开机 LOGO	旧方案
UPDATE_SYSTEM_OTA	0x9091	OTA 升级	
UPDATE_SYSTEM_UBOOT_LOGO	0x9092	更新 uboot logo	旧方案
UPDATE_SYSTEM_KERNEL_LOGO	0x9093	更新 kernel logo	旧方案
UPDATE_SYSTEM_UBOOT_LOGO1	0x9094	更新 uboot logo	新方案
UPDATE_SYSTEM_KERNEL_LOGO1	0x9095	更新 kernel logo	新方案
UPDATE_SYSTEM_ANIMATION	0x9096	更新开机动画	新方案
UPDATE_SYSTEM_WALLPAPER	0x9097	更新壁纸	新方案
EVENT_CONFIG_WIEGAND_DATA	0x10004	韦根配置	
EVENT_SEND_WIEGAND_DATA	0x10005	韦根发送数据	
EVENT_READ_WIEGAND_DATA	0x10006	韦根接收数据	

11.7.1 WiFi 功能开关

命令名称：
SWITCH_WIFI（0x9000）

参数：

参数名	类型	说明
params	String	on：打开 off：关闭

返回值：
--无

代码示例：

```
//打开 WiFi
int[] dataArray = new int[]{0};
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SWITCH_WIFI, "on", dataArray, dataArray.length());
```

备注：
--无

11.7.2 获取 WiFi MAC

命令名称：
GET_WIFI_MAC（0x9001）

参数：

参数名	类型	说明
params	String	无

返回值：
--无

代码示例：

```
//获取 WiFi MAC
int[] dataArray = new int[]{0};
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_WIFI_MAC, params, dataArray, dataArray.length());
```

备注：
--无

11.7.3 蓝牙功能开关

命令名称：
SWITCH_BLUETOOTH（0x9010）

参数：

参数名	类型	说明
params	String	on：打开 off：关闭

返回值：
--无

代码示例：

```
//打开蓝牙
int[] dataArray = new int[]{0};
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_BLUETOOTH_MAC , "on", dataArray, dataArray.length());
```

备注：
--无

11.7.4 以太网功能开关

命令名称：
SWITCH_ETH（0x9020）

参数：

参数名	类型	说明

params	String	方案一：
		on：打开
		off：关闭
		
		方案二：
		true：打开
		false：关闭
		
		方案三：
		json class{
		String ethNum; //网卡名，如：eth0
		boolean flagSwitch; //功能开关
		}

返回值：
--无

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SWITCH_ETH, params, dataArray, dataArray.length());
```

备注：
--无

11.7.5 AP 功能开关

命令名称：
SWITCH_AP（0x9030）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class ApInfo{
    boolean switchONOff;  //功能开关
    String apName;  //AP 名
    boolean apIp;   //指定 IP
    String password; //AP 密码
    boolean offWhenNoDeviceConn;
    /*
    *安全类型
    *SECURITY_TYPE_OPEN = 0
    *SECURITY_TYPE_WPA2_PSK = 1
    *SECURITY_TYPE_WPA3_SAE_TRANSITION = 2
    *SECURITY_TYPE_WPA3_SAE = 3
    */
    private int pwType;
    /*
    *带宽
    *BAND_2GHZ = 1 << 0
    *BAND_5GHZ = 1 << 1
    *BAND_6GHZ = 1 << 2
    *BAND_60GHZ = 1 << 3
    */
    private int band;
}
```

返回值：
--无

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SWITCH_AP, params, dataArray, dataArray.length());
```

备注：
--无

11.7.6 关获取以太网状态

命令名称：
GET_ETH_STATUS（0x9040）

参数：

参数名	类型	说明
params	String	网络驱动名 iface 例：eth0

返回值：

参数名	类型	说明
sVal	String	true：打开 false：关闭

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_ETH_STATUS, params, dataArray, dataArray.length());
```

备注：
--无

11.7.7 设置以太网 IP

命令名称：
SET_ETH_IP（0x9050）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class IpInfo{
    String iface;  //网络驱动名，如 eth0
    boolean isStatic;  //true：静态 IP 模式  false：DHCP
    String ipAddr;
    String mask;
    String gate;
    String dns1;
    String dns2;
}
```

返回值：

参数名	类型	说明
sVal	String	0：成功 其他值：失败

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SET_ETH_IP, params, dataArray, dataArray.length());
```

备注：
--无

11.7.8 获取网络信息

命令名称：
GET_ETH_INFO（0x9060）

参数：

参数名	类型	说明
params	String	网络驱动名 iface 例：eth0 详见“**网络驱动类型参数表**”

返回值：

参数名	类型	说明
sVal	String	网络驱动信息

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_ETH_INFO, params, dataArray, dataArray.length());
```

备注：
--无

11.7.9 获取 AP 信息

命令名称：
GET_AP_INFO（0x9070）

参数：

参数名	类型	说明
params	String	null

返回值：

参数名	类型	说明
sVal	String	AP 驱动信息

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_AP_INFO, params, dataArray, dataArray.length());
```

备注：
--无

11.7.10 设置屏幕饱和度

命令名称：
SET_SCREEN_SATURATION（0x9080）

参数：

参数名	类型	说明
dataArray	int[]	dataArray[0]值范围：0 - 100

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SET_SCREEN_SATURATION, params, dataArray, dataArray.length());
```

备注：
--无

11.7.11 获取屏幕饱和度

命令名称：
GET_SCREEN_SATURATION（0x9080）

参数：

参数名	类型	说明
dataArray	int[]	null

返回值：

参数名	类型	说明
sVal	String	值范围：0 - 100
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_SCREEN_SATURATION, params, dataArray, dataArray.length());
```

备注：
--无

11.7.12 设置屏幕对比度

命令名称：
SET_SCREEN_CONTRAST（0x9082）

参数：

参数名	类型	说明
dataArray	int[]	dataArray[0]值范围：0 - 100

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SET_SCREEN_CONTRAST, params, dataArray, dataArray.length());
```

备注：
--无

11.7.13 获取屏幕对比度

命令名称：
GET_SCREEN_CONTRAST（0x9083）

参数：

参数名	类型	说明
dataArray	int[]	null

返回值：

参数名	类型	说明
sVal	String	值范围：0 - 100
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_SCREEN_CONTRAST, params, dataArray, dataArray.length());
```

备注：
--无

11.7.14 设置屏幕色调

命令名称：
SET_SCREEN_HUE（0x9084）

参数：

参数名	类型	说明
dataArray	int[]	dataArray[0]: 0 - 100

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SET_SCREEN_HUE, params, dataArray, dataArray.length());
```

备注：
--无

11.7.15 获取屏幕色调

命令名称：
GET_SCREEN_HUE（0x9085）

参数：

参数名	类型	说明
dataArray	int[]	null

返回值：

参数名	类型	说明
sVal	String	值范围：0 - 100
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_SCREEN_HUE, params, dataArray, dataArray.length());
```

备注：
--无

11.7.16 更新系统开机 LOGO

命令名称：
UPDATE_SYSTEM_LOGO（0x9090）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class LogoInfo{
    int type;  //0:animation  1:boot logo
    String orientation;   //horizontal:横屏   vertical:竖屏
    String logoFilePath;  //文件存放绝对路径
    int logoFileType;     //0:png  1:zip  2:mp4
}
```

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_LOGO, params, dataArray, dataArray.length());
```

备注：

-- 此接口慢慢废弃，请使用 0x9094

11.7.17 OTA 升级

命令名称：

UPDATE_SYSTEM_OTA（0x9091）

参数：

参数名	类型	说明
params	String	OTA 文件绝对路径 默认路径是 /mnt/sdcard/update.zip

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_OTA, params, dataArray, dataArray.length());
```

备注：

-- 升级前，需确认本地有足够的存储空间

11.7.18 更新 uboot logo

命令名称：

UPDATE_SYSTEM_UBOOT_LOGO （0x9092）

参数：

参数名	类型	说明
params	String	Uboot logo 文件的绝对路径 -- 文件必须是 bmp 格式 -- 格式要求 8bit 或者 24bit 的 bmp 图片 -- boot.bmp 和 boot kernel.bmp 这 2 个图片大小分辨率需保持一致。

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_UBOOT_LOGO, params, dataArray, dataArray.length());
```

备注：

-- 此接口慢慢废弃，请使用 0x9094

11.7.19 更新 kernel logo

命令名称：

UPDATE_SYSTEM_KERNEL_LOGO （0x9093）

参数：

参数名	类型	说明
params	String	kernel logo 文件的绝对路径 -- 文件必须是 bmp 格式 -- 格式要求 8bit 或者 24bit 的 bmp 图片 -- boot.bmp 和 boot kernel.bmp 这 2 个图片大小分辨率需保持一致。

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_KERNEL_LOGO, params, dataArray,
dataArray.length());
```

备注：

-- 此接口慢慢废弃，请使用 0x9095

11.7.20 更新 uboot logo

命令名称：

UPDATE_SYSTEM_UBOOT_LOGO1（0x9094）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class LogoInfo{
    int type;     //0:animation  1:logo  2:wallpaper
    String orientation;  //horizontal:横屏    vertical:竖屏
    String filePath;     //文件存放绝对路径
    int fileType;        //0:image 1:zip  2:video
    int actionType;      //0:import  1:清除已导入文件
}

注意：
-- logo 只支持
  >> bmp 格式文件
  >> 位深 24
  >> 分辨率最大 4K，以板卡实际能力为准
```

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持 iArray[1]: -- 当 iArray[0] = -1 时才可读 -- 详细错误码见“***LOGO* 文件导入错误码表**”。

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_KERNEL_LOGO1, params, dataArray,
dataArray.length());
```

备注：
--无

11.7.21 更新 kernel logo

命令名称：
UPDATE_SYSTEM_KERNEL_LOGO1 （0x9095）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class LogoInfo{
    int type;     //0:animation  1:logo  2:wallpaper
    String orientation;  //horizontal:横屏    vertical:竖屏
    String filePath;     //文件存放绝对路径
    int fileType;        //0:image 1:zip  2:video
    int actionType;      //0:import  1:清除已导入文件
}

注意：
-- logo 只支持
  >> bmp 格式文件
  >> 位深 24
  >> 分辨率最大 4K，以板卡实际能力为准
```

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持 iArray[1]: -- 当 iArray[0] = -1 时才可读 -- 详细错误码见“***LOGO* 文件导入错误码表**”**

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_KERNEL_LOGO1, params, dataArray,
dataArray.length());
```

备注：
--无

11.7.22 更新开机动画

命令名称：
UPDATE_SYSTEM_ANIMATION（0x9096）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class LogoInfo{
    int type;     //0:animation  1:logo  2:wallpaper
    String orientation;  //horizontal:横屏    vertical:竖屏
    String filePath;     //文件存放绝对路径
    int fileType;        //0:image 1:zip  2:video
    int actionType;      //0:import  1:清除已导入文件
}

注意：
-- logo 只支持
  >> zip 和 mp4[h264 或 h265]格式文件
  >> 文件最大 60MB
  >> 分辨率最大 4K，以板卡实际能力为准
```

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持 iArray[1]: -- 当 iArray[0] = -1 时才可读 -- 详细错误码见“***LOGO* 文件导入错误码表**”**

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_ANIMATION, params, dataArray, dataArray.length());
```

备注：
--无

11.7.23 更新壁纸

命令名称：
UPDATE_SYSTEM_WALLPAPER（0x9097）

参数：

参数名	类型	说明
params	String	详见下面的 class 结构 使用时，需要将 class 转成 json string

params 参数 class 结构

```
class LogoInfo{
    int type;     //0:animation  1:logo  2:wallpaper
    String orientation;  //horizontal:横屏    vertical:竖屏
    String filePath;     //文件存放绝对路径
    int fileType;        //0:image 1:zip  2:video
    int actionType;      //0:import  1:清除已导入文件
}

注意：
-- logo 只支持
 >> png 格式文件
 >> 文件最大 10MB
 >> 分辨率最大 5000x5000，以板卡实际能力为准
```

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持 iArray[1]: -- 当 iArray[0] = -1 时才可读 -- 详细错误码见“***LOGO* 文件导入错误码表**”**

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
UPDATE_SYSTEM_WALLPAPER, params, dataArray, dataArray.length());
```

备注：
--无

11.7.24 设置屏幕信号亮度

命令名称：
SET_SCREEN_SIGNAL_BRIGHTNESS（0x9086）

参数：

参数名	类型	说明
dataArray	int[]	dataArray[0]值范围：0 - 100

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
SET_SCREEN_SIGNAL_BRIGHTNESS, params, dataArray,
dataArray.length());
```

备注：
--无

11.7.25 获取屏幕信号亮度

命令名称：
GET_SCREEN_SIGNAL_BRIGHTNESS（0x9087）

参数：

参数名	类型	说明
dataArray	int[]	null

返回值：

参数名	类型	说明
sVal	String	值范围：0 - 100
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
GET_SCREEN_SIGNAL_BRIGHTNESS , params, dataArray,
dataArray.length());
```

备注：
--无

11.7.26 韦根配置

命令名称：
EVENT_CONFIG_WIEGAND（0x10004）

参数：

参数名	类型	说明
params	String	韦根 bit 位，内容是“26”、“32”、“34”、“58”
dataArray	int[]	需初始化 1 字节大小数据，内容是 0

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]： 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
EVENT_CONFIG_WIEGAND, params, dataArray, dataArray.length());
```

备注：
--无

11.7.27 韦根发送数据

命令名称：
EVENT_SEND_WIEGAND_DATA（0x10005）

参数：

参数名	类型	说明
dataArray	int[]	-- int 数组：根据韦根 bit 位，传固定大小数据 -- 例：26 位模式下，传 3 字节数据：dataArray[3]={0x01, 0x02, 0x03}

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: 0：成功 1：失败 2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
EVENT_SEND_WIEGAND_DATA , params, dataArray, dataArray.length());
```

备注：
--无

11.7.28 韦根接收数据

命令名称：
EVENT_READ_WIEGAND_DATA（0x10006）

参数：

参数名	类型	说明
dataArray	int[]	-- int 数组：需初始化 1 字节大小数据，内容是 0 建议：每隔 500ms，读取一次韦根数据

返回值：

参数名	类型	说明
iArray	int[]	iArray[0]: -- 0：成功 >> strVal:返回读到的数据，否则返回失败 -- -1：失败 -- -2：接口不支持

代码示例：

```
SdkApi.getInstance().CommonEvent().setCommonEvent(AppId,
EVENT_READ_WIEGAND_DATA , params, dataArray, dataArray.length());
```

备注：
--无

API 常量

LOGO 文件导入错误码表

变量	码值	说明
FILE_NOT_EXIST	0x0001	文件不存在
FILE_NOT_READABLE	0x0002	文件不可读
GSON_FORMAT_FAIL	0x0003	String 转义 Json 失败
INVALID_GSON_INFO	0x0004	数据不符合 LOGO class 定义
INVALID_ORIENTATION	0x0005	方向参数字段不匹配
INVALID_SOURCE_FILE_PATH	0x0006	路径参数字段为空
NO_ENOUGH_CACHE_SIZE	0x0007	目标目录存储空间不够
INVALID_FILE_SUFFIX	0x0008	无效的文件后缀
FILE_COPY_FAIL	0x0009	文件拷贝失败
FILE_SIZE_OVER_LIMIT	0x000A	文件大小超过限制
BMP_HEADER_SIZE_INVALID	0x1010	BOOT 图片文件头大小不对
BMP_SIGNATURE_INVALID	0x1012	BOOT 图片校验失败
BMP_FILE_SIZE_NOT_MATCH	0x1013	BOOT 图片文件大小不匹配
BMP_BIT_DEPTH_UNSUPPORTED	0x1014	BOOT 图片位深不支持
BMP_DECODE_FAIL	0x1015	BOOT 图片解码失败

BMP_RESOLUTION_NOT_SUPPORT	0x1016	BOOT 图片分辨率不支持
ANIMATION_MP4_SUFFIX_INVALID	0x1020	MP4 动画文件后缀不支持
ANIMATION_MP4_HEADER_INVALID	0x1021	MP4 动画文件头格式不对
ANIMATION_MP4_DECODE_FAIL	0x1022	MP4 动画文件解码失败
ANIMATION_MP4_FORMAT_UNSUPPORTED	0x1023	MP4 动画文件校验失败
ANIMATION_ZIP_FORMAT_NOT_STORED_ONLY	0x1024	ZIP 动画文件不是 Stored 压缩
ANIMATION_ZIP_DESC_NOT_FOUND	0x1025	ZIP 动画文件内描述文件没找到
ANIMATION_ZIP_DESC_INVALID	0x1026	ZIP 动画文件内描述文件无效
ANIMATION_ZIP_PNG_INVALID	0x1027	ZIP 动画文件内不是 PNG 文件
ANIMATION_MP4_RESOLUTION_NOT_SUPPORT	0x102A	MP4 动画文件分辨率不支持
ANIMATION_ZIP_DECODE_FAIL	0x102B	ZIP 动画文件解码失败
WALLPAPER_DECODE_FAIL	0x1030	壁纸图片解码失败
WALLPAPER_FORMAT_UNSUPPORTED	0x1031	壁纸图片格式不支持
WALLPAPER_RESOLUTION_NOT_SUPPORT	0x1032	壁纸图片分辨率不支持

网络驱动类型参数表

变量	码值	说明
NET_TYPE_ETH0	eth0	以太网网卡 1
NET_TYPE_ETH1	eth1	以太网网卡 2

NET_TYPE_WIFI	wifi	WiFi
NET_TYPE_MOBILE	mobile	移动网络

GPIO 值定义

组号定义如下

```
    public static final int GPIO0 = 0;
    public static final int GPIO1 = 1;
    public static final int GPIO2 = 2;
    public static final int GPIO3 = 3;
    public static final int GPIO4 = 4;
    public static final int GPIO5 = 5;
    public static final int GPIO6 = 6;
    public static final int GPIO7 = 7;
    public static final int GPIO8 = 8;
```

IO 序号定义如下

```
    public static final int PA0 = 0;
    public static final int PA1 = 1;
    public static final int PA2 = 2;
    public static final int PA3 = 3;
    public static final int PA4 = 4;
    public static final int PA5 = 5;
    public static final int PA6 = 6;
    public static final int PA7 = 7;

    public static final int PB0 = 8;
    public static final int PB1 = 9;
    public static final int PB2 = 10;
    public static final int PB3 = 11;
    public static final int PB4 = 12;
    public static final int PB5 = 13;
    public static final int PB6 = 14;
    public static final int PB7 = 15;

    public static final int PC0 = 16;
    public static final int PC1 = 17;
    public static final int PC2 = 18;
    public static final int PC3 = 19;
    public static final int PC4 = 20;
    public static final int PC5 = 21;
    public static final int PC6 = 22;
    public static final int PC7 = 23;

    public static final int PD0 = 24;
    public static final int PD1 = 25;
    public static final int PD2 = 26;
    public static final int PD3 = 27;
    public static final int PD4 = 28;
    public static final int PD5 = 29;
    public static final int PD6 = 30;
    public static final int PD7 = 31;
```